

Analyzing the Effect of Network Width on Graph Transformers

Progress report — what was done, what came out, and what it means

Weeks 1–5 of the department calendar · 2026-09-24 → 2026-10-09 · raw log: [docs/CHANGELOG.md](#) · questions: [docs/WIDTH-QUESTIONS.md](#)

Summary.

1. **Width has to be measured with the learning rate tuned per width.** With one fixed rate, wider models look up to 0.4 worse than they are; the best rate falls as width grows.
2. **Width has a floor and a data-dependent ceiling.** Below a minimum width the model cannot fit at all; above some width it memorizes, and that ceiling moves up with more training data (2000 → 8000 graphs removes it).
3. **Our first benchmark was too easy.** It leaked component membership through node indices and graph statistics. We built an audited replacement (*swap*) on which statistics and short-range checks are at chance.
4. **On the clean task, width decides whether learning starts at all.** At $n = 64$ only $m \geq 64$ leave the trivial predictor; at $n = 128$ nothing up to $m = 128$ does.
5. **The width needed to learn connectivity grows superlinearly in n — about $n^{1.9}$ over $n = 32$ – 56 — even with ample data.** With 32 000 graphs, fitting and generalizing come apart ($n^{1.55}$ vs $n^{2.27}$); with 128 000 graphs they coincide, and the shared critical width grows as $n^{1.94}$ at one learning rate, $n^{1.85}$ with the rate tuned where tested (seed-bootstrap 90 % range 1.68–2.03). Doubling the training steps does not lower it, and neither does a fixed-width input encoding (Q6). Theory says this depth needs far less width just to **represent** connectivity.
6. **Depth saves little width at small n , but much more as n grows.** At $n = 40$, two layers learn connectivity at width $\approx 1.1n$ (45) and six layers at $\approx 0.75n$ (30); beyond 4–6 layers depth buys nothing. A 2-layer model solves graphs of diameter ≈ 10 — it does not follow paths hop by hop. But across sizes the 2-layer critical width grows as $\approx n^3$, not linearly as the constant-depth construction allows: at $n = 56$ two layers need about $3\times$ the width of six (fit 168 vs 53) and do not generalize to 0.95 by width 192 (Q7c).

1. Question and set-up

Thesis question. How does the width of a graph transformer — its embedding dimension m — determine what graph problems it can solve and learn, and how does that need scale with graph size n ?

Task connectivity matrix. Input: a graph on n nodes. Output: the $n \times n$ matrix R with $R_{ij} = 1$ iff i and j are in the same component. Dense per-pair supervision; a single connected/disconnected bit per graph does not train (stalls at $\ln 2$, June results). Task, tokens and read-out are taken from Ye et al. (2026), who study which solution this model learns.

Model transformer encoder whose tokens are the rows of $A + I$ (one token per node), pre-norm blocks, multi-head attention with head dimension fixed at 8 (so heads = $m/8$), pairwise bilinear read-out $HWH^\top$.

Width m , the embedding (residual-stream) dimension. Every part of the model scales with it: attention uses $m/8$ heads of dimension 8 (so heads $\times$ head size = m , the product mH that the theory bounds), and each feed-forward layer is $4m$ wide. Depth L = number of blocks.

Metrics **exact-match** (the whole matrix right) and **pair accuracy** (fraction of entries right). The trivial “everything connected” predictor scores 0.5 exact-match (half the graphs are connected) and ≈ 0.75 – 0.77 pair accuracy.

Protocol train / validation / test split; learning rate and checkpoint picked on **validation**, reported on test; 3 seeds unless stated. **Critical width** m^* = the smallest m at which at least 2/3 of seeds reach the threshold (0.90 / 0.95 / 0.99).

Compute sweeps run on Kaggle ($2 \times \text{T4}$) as 6–8 parallel shards (`width_sweep.py`, `kaggle/width_sweep.ipynb`); every run is one JSON line in `results/width/`.

2. Why these numbers

Every number in the set-up is either **principled** (follows from the question or a measurement), **checked** (chosen, then tested and shown not to drive the result), **inherited** (the default of the June connectivity code this work started from, kept for continuity), or **budget** (set by Kaggle’s 30 GPU-hours a week). The tags below say which.

The model and what “width” changes

Choice	Value	Why
width m	embedding dim	The residual-stream dimension is what every theory bound calls width (the m in mHp) and what “model width” means for transformers. One knob that makes the whole model wider in proportion. principled
head size	8	Fixed so that only width moves: a wider model gets more heads, not bigger ones. 8 is the largest size that still gives the narrowest model ($m = 8$) one head; the literature sweep flagged that changing head size together with width confounds the two (Q5 would vary them separately). principled
heads H	$m/8$	Follows from the fixed head size. Heads $\times$ head size $= m$, so our width equals the product mH in Sanford et al.’s bounds. principled
feed-forward	$4m$, ReLU	The standard transformer ratio (Vaswani et al. 2017), kept proportional so it scales with m rather than being a second width. inherited
blocks	pre-norm	Pre-norm residual blocks train stably without careful initialisation. inherited
input	adjacency row $A + I$	One token per node carrying its neighbour set — the tokenization of Yehudai et al. 2025, so their linear-width result is the comparison point. Q6 replaced it with a fixed 96-wide code and the scaling held. checked
read-out	$HWH^\top$	Scores every node pair, giving the $n \times n$ connectivity matrix directly; symmetrised because connectivity is symmetric. inherited
dropout, weight decay	0, 0	In the July chain-of-thought experiments weight decay alone blocked circuit formation (it pushes attention towards low rank) and dropout gave no benefit; both were left off here and not re-tested on this task. inherited

Training

Choice	Value	Why
optimiser	Adam	Standard for transformers; no weight decay (above). inherited
schedule	5 % warm-up, cosine	Q1 showed wide models diverge at high rates without warm-up; Q1b added it and the instability went away. checked
learning rate	tuned per width	Q1: one fixed rate fakes a width effect of up to 0.4; the best rate falls as width grows. Grids: $3 \cdot 10^{-4} \dots 3 \cdot 10^{-2}$ in Q1/Q1b, where the best rate fell from 10^{-2} – $3 \cdot 10^{-2}$ at small widths to 10^{-3} at $m = 256$. Q3 used $3 \cdot 10^{-3}$ (the only rate that started learning at $n = 64$) and then re-checked with 10^{-3} , which helped only the widest models at $n = 56$. checked
batch size	128	Large enough for stable gradients, small enough for many steps per epoch; never varied. inherited
steps	4 800 $\rightarrow$ 24 000	Held fixed across data sizes so more data never means more training. 4 800 is Q1’s 300 epochs $\times$ 16 batches; 24 000 is where the learnability probe had converged (it was at 0.95–0.98 halfway through 24 000); 48 000 in the step check moved nothing. checked
evaluation	≈ 15 times per run	Enough points for learning curves; training accuracy on the first 8 000 training graphs only, so evaluation stays cheap at 128 000. budget
checkpoint	final epoch	Choosing the “best” epoch by validation exact-match picks epoch 1 when exact-match never moves, hiding real learning (the 10-05 correction). The cosine schedule makes the final epoch a fair choice. checked

Data

Choice	Value	Why
generator	swap	The only one of five candidates on which every heuristic we could build — graph statistics, ≤ 4 -hop checks, node indices — sits at chance. principled
chords	$\frac{1}{4} \times$ blob size	Sparse enough that graphs are not locally solvable, dense enough that diameter grows only $\approx \log n$ (a pure cycle’s diameter grows $\approx n/2$, and nothing learned at $n \geq 32$ in pilot 1). checked
$d_{\min}$	6 hops	Every cycle the two swapped edges close has ≥ 7 edges, so closed-walk counts up to length 6 cannot see the label. With 5, statistics still predicted the label at $n = 32$ (0.63). Cost: graphs below $n \approx 32$ cannot be built. checked
blob split	$n_a \sim U[n/4, 3n/4]$	Avoids tiny blobs that are trivially dense or cannot hold a far-apart pair. principled
training graphs	2 000 $\rightarrow$ 128 000	Raised until the data ceiling disappeared: at 32 000, fitting and generalizing still came apart; at 128 000 they coincide. 2 000 was the June default. checked
val / test	400 / 400	Separate seeds; 400 graphs give $\approx 110\,000$ – $620\,000$ scored node pairs each ($n = 24$ – 56) — far more than needed for pair accuracy. inherited

Measurement

Choice	Value	Why
main metric	pair accuracy	Exact-match needs all $\approx n^2/2$ pairs right, so it gets stricter as n grows and would build a fake n -dependence into the result. Pair accuracy is comparable across n . principled
threshold	0.95	Well above the trivial ≈ 0.75 and below the ≈ 0.99 the best models reach, so the crossing sits on the steep part of the curve. 0.90 and 0.99 were also reported in Q1–Q3; they shift m^* but not the conclusions. checked
m^*	per-seed crossing	Interpolated in $\log_2 m$ between grid widths, then averaged over seeds — sturdier than crossing the averaged curve when one seed dips at a single width. principled
seeds	2–3	Three in the early and fine grids; two in the large-data checks, where seeds agreed within $\approx \pm 3$ and runs cost most. budget

Sweep grids

Choice	Value	Why
widths	16 ... 192	Powers of two, with in-between points (24, 48, 96) near each threshold so the crossing is located to within $\approx 20\%$. Later sweeps placed the grid around each size's or depth's expected threshold to spend no runs far from it. budget
graph sizes	$n = 32 \dots 56$	The range where m^* can be measured: swap cannot be built below ≈ 32 , and at $n \geq 64$ nothing learned within affordable width. $n = 24$ in Q1 was the June default. budget
depth	$\lceil \log_2 n \rceil$, fixed 6	Theory's depth for connectivity; fixed at 6 — which is $\lceil \log_2 n \rceil$ for every n in 33–64 — so depth does not step up between sizes. Q7a varied it from 2 to 8. principled
projected input	$k = 96$	The smallest code width that keeps each neighbour set recoverable ($\approx 99\%$ at $n = 32$ and 56); $k = 32$ recovered only 60–70 %, which would have confused width with lost information. checked

! Caveat

The **inherited** and **budget** choices (feed-forward ratio, batch size, dropout and weight decay, number of seeds, the n range) were not varied; head size is fixed by design and is what Q5 would vary. The checked ones were, and none of them changed the main result.

3. Timeline

Date	Step	Outcome
09-24	Topic settled; literature sweep	35 papers, theory predictions (docs/WIDTH-LITERATURE.md)
09-30	Q1: fixed vs tuned LR (72 runs)	fixed LR distorts the width curve
10-01	Q1b: + warm-up/cosine (120 runs)	instability fixed; wide-model drop remains
10-01	Q1c: width $\times$ data (216 runs)	the drop is memorization; data moves the ceiling
10-01	Found the index leak in <code>hard</code>	Q1 measures a near-local task
10-02	Q3 pilot 1: <code>hard_diam</code> (18 runs)	diameter $\approx n/2$; nothing learns at $n \geq 32$
10-02	Shortcut audit; <code>swap</code> generator	statistics at chance, ≤ 4 -hop checks at 0
10-02	Q3 pilot 2: <code>swap</code> , 8000 graphs	17 of 18 runs stay trivial
10-02	Learnability probe (6 runs)	learns to 0.98–0.998 with 32 000 graphs
10-04	Q3 trimmed grid (60 runs)	clean floor at $n = 32$; onset width grows with n
10-05	Q3 fine grid ($n = 32$ –56, 84 runs)	critical width $\propto n^{1.55}$ (fit), $n^{2.27}$ (generalize)
10-05	Q3 data check (32 runs)	128 000 graphs close the fit/generalize gap
10-06	Q3 at 128k, $n = 32 / 40$ (24 runs)	width to learn $\propto n^{1.94}$ with enough data
10-07	Q3 step budget: $2\times$ steps (16 runs)	critical width unchanged — not a training-time effect
10-07	Q3 learning rate: 10^{-3} (16 runs)	helps wide models at $n = 56$; slope $n^{1.94} \rightarrow n^{1.85}$
10-07	Q6: fixed 96-wide input (24 runs)	growth persists ($n^{2.0}$) — not the read-in
10-08	Q7a: depth 2–8 $\times$ width at $n = 40$ (60 runs)	depth 2 learns at $m \approx 1.1n$; depth saves $\approx \frac{1}{3}$ of the width
10-09	Q7c: depth 2 at $n = 32 / 48 / 56$ (34 runs)	depth-2 critical width $\propto n^{3.1}$ (fit), not linear

Table 1: What was done, in order.

4. Q1 — the learning rate confounds width

Set-up. $n = 24$, depth 2, 2000 training graphs, widths 8–256, learning rates $3 \cdot 10^{-4}$... 10^{-2} , 3 seeds (Q1). Q1b repeats it with 5 % linear warm-up and cosine decay, adds widths 2 and 4 and rate $3 \cdot 10^{-2}$.

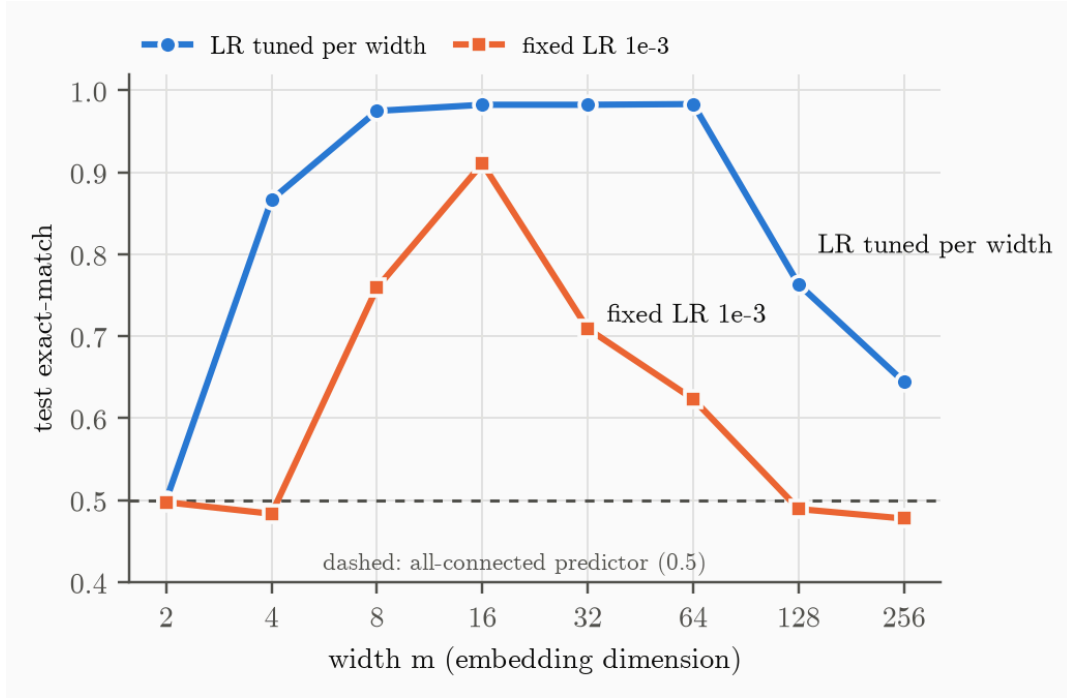


Figure 1: Q1b: test exact-match vs width with one fixed learning rate versus the rate chosen per width on validation (mean of 3 seeds).

m	2	4	8	16	32	64	128	256
Q1, fixed 1e-3	–	–	0.917	0.917	0.781	0.912	0.820	0.596
Q1, tuned	–	–	0.976	0.977	0.967	0.948	0.844	0.678
Q1b, fixed 1e-3	0.497	0.483	0.759	0.911	0.709	0.623	0.489	0.478
Q1b, tuned	0.497	0.867	0.975	0.967	0.982	0.983	0.763	0.644

Table 2: Test exact-match, mean of 3 seeds.

✓ Finding

- With a fixed rate the width curve zig-zags and collapses (Q1b: 0.48 at $m \geq 128$); tuned per width, $m = 8\text{--}64$ all reach $\approx 0.97\text{--}0.98$.
- The best rate **falls as width grows** (10^{-2} at small $m \rightarrow 10^{-3}$ at $m = 256$); without warm-up, wide models diverge at high rates ($m = 256$ at 10^{-2} : train 0.41).
- Warm-up fixes the instability (wide models reach train 1.00) — but tuned test at $m = 128/256$ stays at 0.76 / 0.64.

5. Q1c — the wide-model drop is memorization

Same set-up with 500, 2000 and 8000 training graphs, at a **fixed number of optimizer steps** (4800), so more data never means more training.

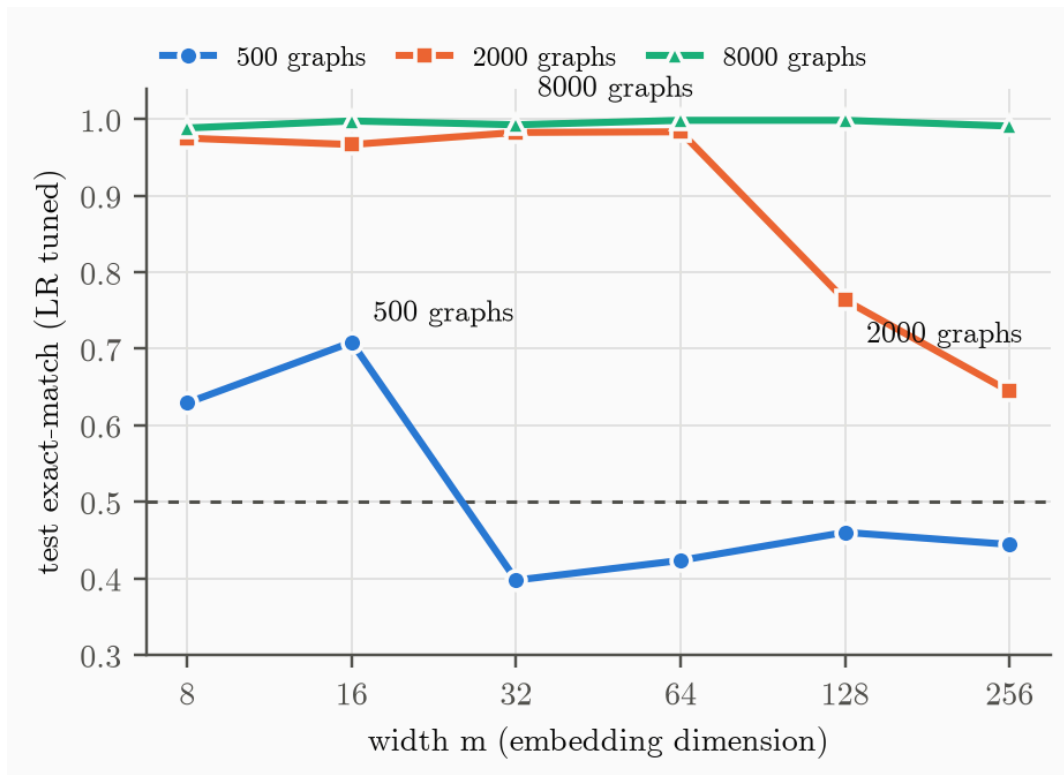


Figure 2: Q1c: tuned test exact-match vs width for three training-set sizes. Training accuracy is ≈ 1.00 everywhere.

✓ Finding

- At 8000 graphs every $m \geq 16$ reaches ≥ 0.99 — the drop at $m \geq 128$ is gone.
- At 500 graphs everything memorizes; $m \geq 32$ falls **below** the trivial predictor.
- The 2000-graph arm reproduces Q1b to the third decimal (independent Kaggle run).
- With enough data the fixed rate is within 0.01 of tuned; tuning matters most when data is scarce.

◆ What it means

Width has a **floor** (below ≈ 4 – 8 the model cannot even fit the training set) and a **ceiling set by the data** (past it, extra width memorizes). The floor is what theory talks about; the ceiling is the practical limit it doesn't cover.

! Caveat

Q1 used the **hard** generator, which turned out to leak (next section). Q1's lessons are about **method** — tune the rate per width, give enough data — not about reachability.

6. Making the task honest: the shortcut audit

The **hard** generator builds two dense blobs, with or without one bridge. Three problems surfaced:

1. **Index leak.** Node labels were never shuffled: the blobs are always nodes $[0, n_a)$ and $[n_a, n)$ (200 / 200 graphs checked). With adjacency-row tokens the split point is readable, so connectivity reduces to “does any row have ones on both sides”.
2. **Statistics leak.** A bridge closes no cycle; the matched extra edge in the disconnected class always closes a short one. Gradient boosting on degree and closed-walk counts ($\text{tr } A^3 \dots \text{tr } A^6$) predicts the label with 0.68–0.90 accuracy at $n = 16$ for every existing generator.
3. **Locality.** On small graphs a model that only looks 6 hops away gets the whole matrix right on 60–100 % of graphs.

Fix — the swap generator. Two sparse blobs (cycle + $\frac{1}{4} \cdot \text{size}$ chords). Pick two nodes ≥ 6 hops apart in each blob (a_1, a_2 and b_1, b_2). Disconnected: add $a_1 a_2$ and $b_1 b_2$. Connected: add $a_1 b_1$ and $a_2 b_2$. Same degree changes in both classes; every new cycle has ≥ 7 edges; graphs failing the distance test are redrawn **before** the label is applied; node labels shuffled per graph. The idea — classes that no local statistic can tell apart, two cycles vs one — is the “cycle task” Abbe et al. (NeurIPS 2024) proposed as a training benchmark; **swap** adds chords (so diameter grows only $\approx \log n$), a dense pair target, and an explicit audit.

n	32	40	48	56	64	128
statistics $\rightarrow$ label	0.526	0.512	0.493	0.504	0.512	0.507
statistics + 4-hop (exact)	0.256	0.257	0.246	0.256	0.259	0.253
4-hop check (exact)	0.000	0.000	0.000	0.000	0.000	0.000
diameter, connected	8.6	9.8	10.9	11.7	12.5	16.7

Table 3: Audit of **swap** (chord fraction 0.25, $d_{\min} = 6$) — `audit_width_data.py`. Trivial predictor: 0.5. Diameter grows $\approx \log n$.

◆ What it means

Every heuristic we could think of sits at or below the trivial predictor, so a model that scores well on **swap** has to trace paths. The price: **swap** cannot be built below $n \approx 32$, and it is much harder to learn.

7. Q3 — critical width versus graph size

Pilots and the learnability probe

Run	Set-up	Result
Pilot 1	<code>hard_diam</code> (diameter $\approx n/2$), $n = 16/32/64$, 8000 graphs	$n = 16$ learns (depth 4: 0.99); $n \geq 32$ all at 0.50
Pilot 2	swap , $n = 32/64/128$, 8000 graphs, 4800 steps	17 / 18 at 0.50; only $n = 32$, depth 5, $m = 128$: pair 0.87 vs 0.75 trivial
Probe	swap , $n = 32$, depth 5, 32 000 graphs, 24 000 steps	$m = 64 / 128 / 256$ reach 0.985 / 0.998 / 0.988 test exact-match

Table 4: The road to a learnable, shortcut-free setting.

✓ Finding

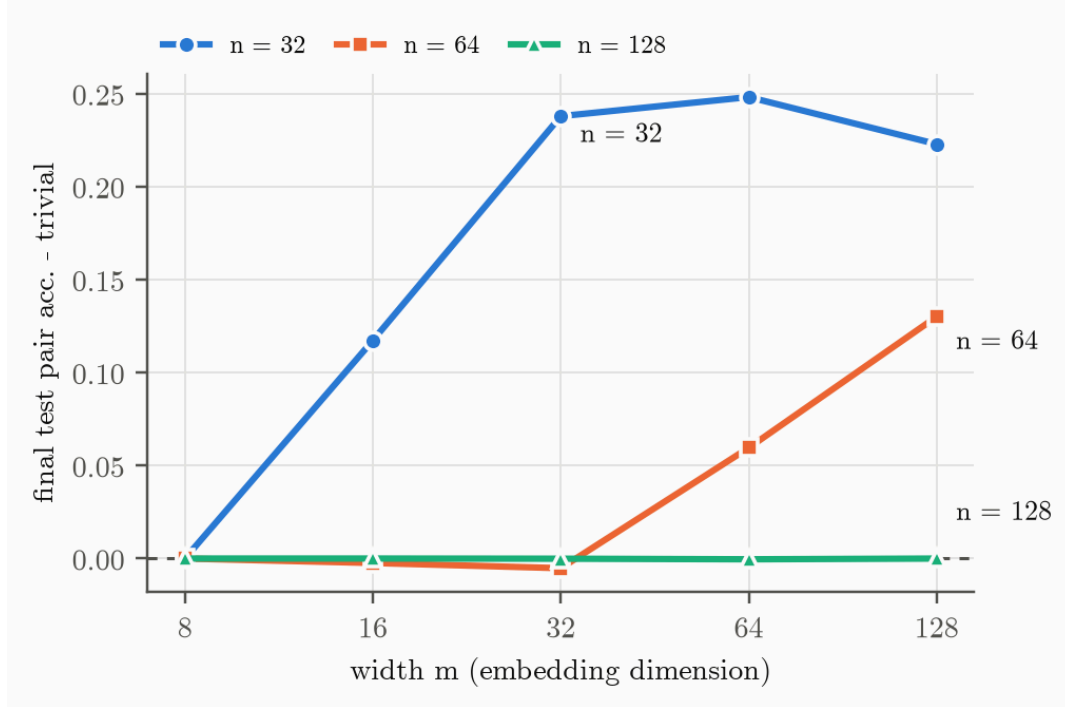
The clean task **is** learnable — test tracks train (0.998 vs 1.000), pair accuracy 0.998. Pilot 2 failed mainly for lack of data (with 8000 graphs the $m = 128$ model memorized: train 0.99, test 0.53), the same data ceiling as Q1c. Wider models also start later at a fixed rate (rate $3 \cdot 10^{-3}$, epoch 12: 0.83 / 0.40 / 0.07 for $m = 64 / 128 / 256$).

The trimmed grid

swap, $n = 32 / 64 / 128$ at depth $\lceil \log_2 n \rceil = 5 / 6 / 7$, $m = 8 \dots 128$, 32 000 graphs, 12 000 steps, rates 10^{-3} and $3 \cdot 10^{-3}$, 2 seeds (60 runs).

m	$n = 32$	$n = 64$	$n = 128$
8	0.50 (train 0.27)	0.50	0.50
16	0.49 (train 0.39)	0.50	0.50
32	0.90	0.50	0.50
64	0.97	0.50	0.50
128	0.97	0.50	0.50

Table 5: Tuned test exact-match (mean of 2 seeds).

Figure 3: How far each model gets above the trivial predictor (final test pair accuracy at rate $3 \cdot 10^{-3}$, mean of 2 seeds, no selection on test). The width at which learning starts moves right as n grows.

✓ Finding

- $n = 32$: a sharp floor. $m = 8$ cannot fit the training set; $m = 16$ starts but stalls; $m = 32$ reaches 0.90; $m \geq 64$ reach 0.97. **m^* is between 32 and 64** (by pair accuracy: 32 at ≥ 0.95 , 64 at ≥ 0.99).
- $n = 64$: $m \leq 32$ never leave the trivial predictor; $m = 64$ starts (train loss $0.59 \rightarrow 0.07$ – 0.10 , pair 0.80 – 0.85); $m = 128$ gets further (pair 0.89 – 0.90). Exact-match stays 0.5 — about 4000 pairs must all be right.
- $n = 128$: nothing moves for any $m \leq 128$.

◆ What it means

The width at which learning **begins** is ≈ 16 at $n = 32$, ≈ 64 at $n = 64$ and above 128 at $n = 128$ — growing at least linearly in n . This is the direction of the theory: a single step that gathers a node’s whole reach set is q -sparse averaging with q close to n , which needs width of order n (Sanford, Hsu, Telgarsky 2023; see [reports/qsas-sparse-averaging.pdf](#)). On the old, leaky task the same model needed only $m = 8$ at $n = 24$ — the shortcuts were hiding the width requirement.

The fine grid: a scaling law

swap, $n = 32 / 40 / 48 / 56$ at a fixed depth 6 ($= \lceil \log_2 n \rceil$ for all four), $m = 16 \dots 128$, 32 000 graphs, 24 000 steps, rate $3 \cdot 10^{-3}$, 3 seeds (84 runs).

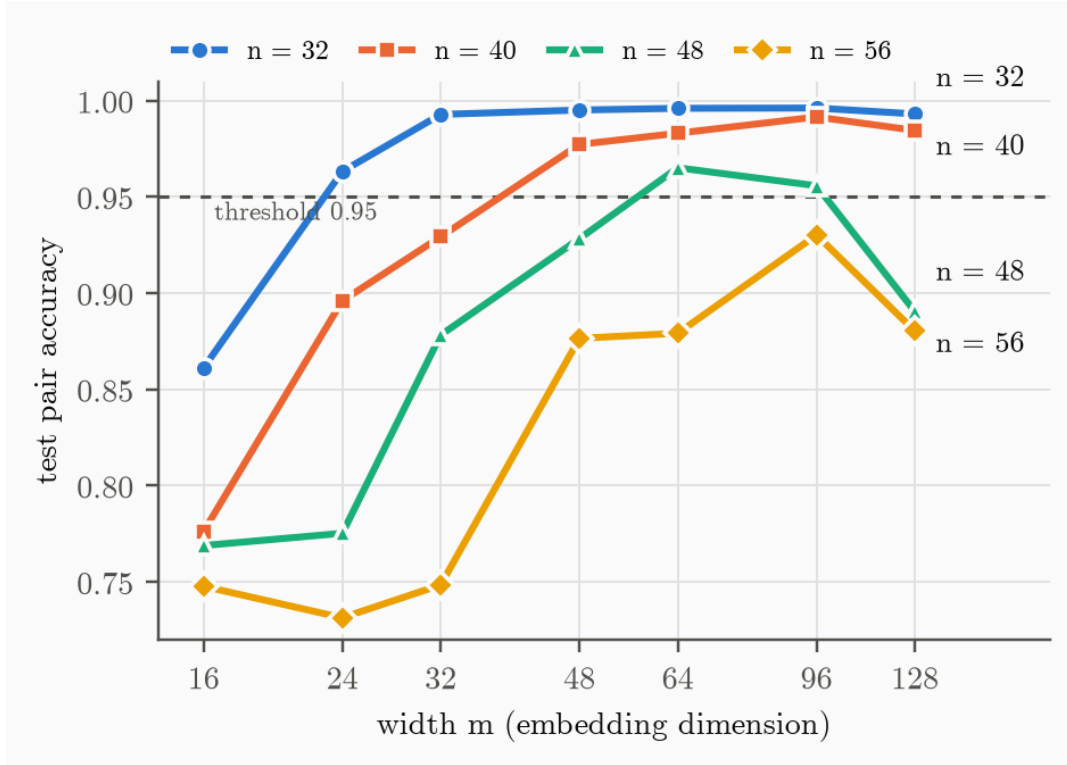


Figure 4: Final-epoch test pair accuracy vs width (mean of 3 seeds; trivial ≈ 0.75). Each curve shifts right as n grows. The drop at $m = 128$ is the learning-rate effect from Q1: $3 \cdot 10^{-3}$ is too high there, and training accuracy falls too.

	$n = 32$	$n = 40$	$n = 48$	$n = 56$	growth
width to fit (final train pair ≥ 0.95)	18.6	23.5	31.7	44.7	$\propto n^{1.55}$
width to generalize (final test pair ≥ 0.95)	22.9	37.6	57.5	> 96	$\propto n^{2.27}$

Table 6: Critical width: mean over seeds of the width where each seed’s curve crosses 0.95 (interpolated in $\log_2 m$); slope of the log–log fit across n ($m \leq 96$; `width_sweep.py q3fine --scaling`). Per-seed crossings agree within $\approx \pm 3$.

! Caveat

Correction (2026-10-05). An earlier version of this report read test pair accuracy at the checkpoint with the best validation **exact-match**. When exact-match never rises above 0.5, every epoch ties and epoch 1 is chosen, so the trained model’s pair accuracy was replaced by the trivial predictor’s (e.g. $n = 48$, $m = 32$: reported 0.755, actually 0.885 / 0.853). All pair-accuracy numbers now use the final epoch. The critical widths barely move (generalize slope $2.19 \rightarrow 2.27$); the curves figure changed most. New runs also log validation pair accuracy.

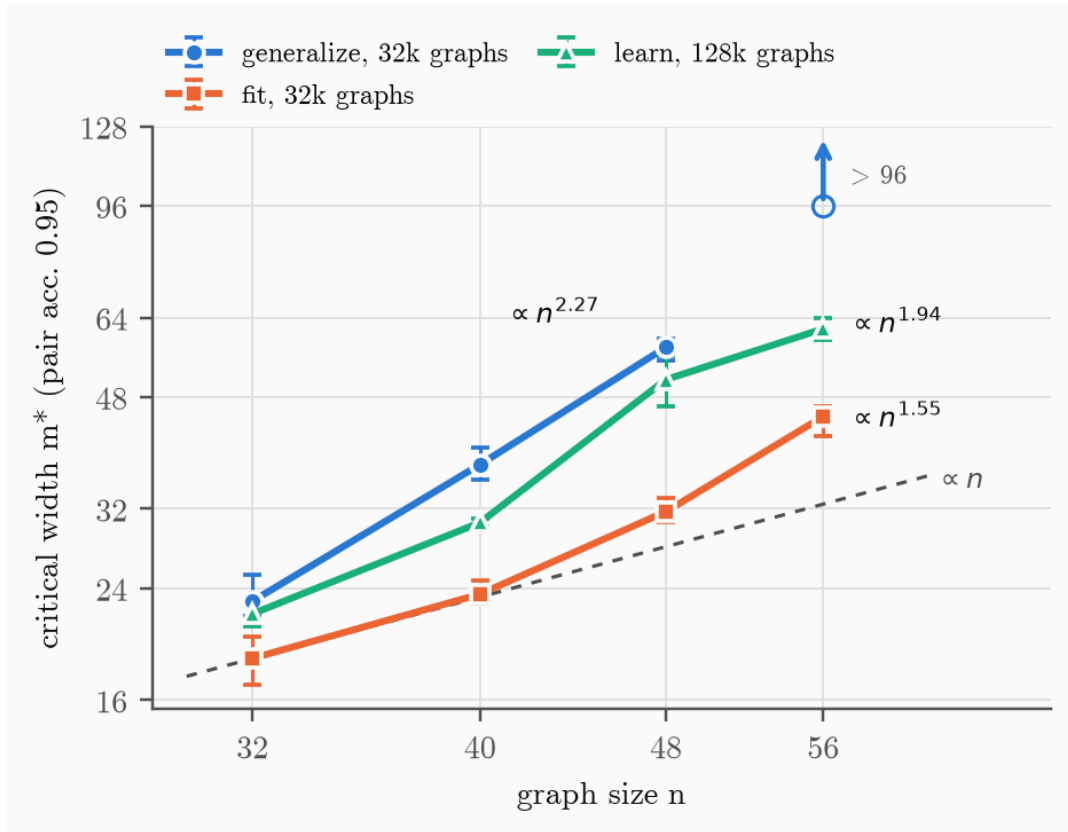


Figure 5: Critical width vs graph size, log-log (error bars: range over seeds). With 32 000 graphs, fitting and generalizing come apart; with 128 000 graphs (green; data check and the $n = 32 / 40$ runs below) they coincide, and the width to learn still grows as $n^{1.94}$ — steeper than $m \propto n$ (dashed).

✓ Finding

- Both critical widths grow **faster than linearly** in n : $\approx n^{1.5}$ to fit, $\approx n^{2.3}$ to generalize.
- The gap between them widens with n : at $n = 56$, $m = 64$ fits the training set (pair 0.984) but reaches only 0.879 on test.
- Seeds agree closely; the scaling is not noise.

◆ What it means

Theory says depth $\log N$ needs width only just above $\sqrt{N}$ to **represent** connectivity, with no extra tokens ($N = \text{vertices} + \text{edges in their encoding}$; Sanford et al. 2024a, Thm 18 — width N^ϵ for small ϵ needs $\approx N^{4-\epsilon}$ extra “pause” tokens, which our models do not have). What we measure is the width to **learn** it, and that grows faster than n . The widening fit/generalize gap is the Q1c data ceiling again: 32 000 graphs give fewer examples per pair as n grows, so the 2.27 is probably inflated by holding data fixed. The data check tests this.

The data check: more data closes the gap

Same set-up at $n = 48 / 56$, $m = 32 \dots 96$, with 64 000 and 128 000 graphs at the same 24 000 steps (2 seeds). The larger training sets start with the same graphs as the smaller ones; validation and test sets are identical.

	critical width	32k graphs	64k graphs	128k graphs
$n = 48$	to fit	31.7	40.5	48.4
	to generalize	57.5	54.1	51.1
$n = 56$	to fit	44.7	57.8	59.4
	to generalize	> 96	$> 96^\dagger$	61.5

Table 7: Critical width (mean of per-seed crossings at 0.95, final epoch). 32k: 3 seeds; 64k / 128k: 2 seeds. $\dagger$ one seed 81.8, the other above 96.

✓ Finding

- **The gap closes.** At 128 000 graphs the widths to fit and to generalize nearly meet: 48 vs 51 at $n = 48$, 59 vs 61 at $n = 56$. At $n = 56$, $m = 64$: train / test pair 0.984 / 0.879 with 32k graphs $\rightarrow$ 0.975 / 0.927 with 64k $\rightarrow$ 0.972 / 0.960 with 128k.
- **Fitting gets harder with more data at fixed steps** (each graph is seen fewer times), so the fit width rises: 31.7 $\rightarrow$ 48.4 at $n = 48$.
- At 128k the width to **learn** is ≈ 51 and ≈ 61 for $n = 48$ and 56, about $1.1n$.

◆ What it means

The widening gap at 32 000 graphs was the data ceiling: with enough data, fitting and generalizing measure the same thing — the width to **learn** connectivity. Two sizes 17 % apart cannot fix a slope, so the same 128 000-graph runs were repeated at $n = 32 / 40$.

Four sizes with enough data

$n = 32 / 40$ at 128 000 graphs, $m = 16 \dots 96$, same 24 000 steps and rate, 2 seeds (24 runs), joined with the data check’s $n = 48 / 56$.

	$n = 32$	$n = 40$	$n = 48$	$n = 56$	growth
width to fit	20.9	29.5	48.4	59.4	$\propto n^{1.95}$
width to generalize	21.9	30.4	51.1	61.5	$\propto n^{1.94}$
generalize m^*/n	0.68	0.76	1.07	1.10	

Table 8: Critical widths at 128 000 graphs (mean of per-seed crossings at 0.95, final epoch). Resampling one seed per n gives a 90 % range of 1.74–2.14 for the generalize slope.

✓ Finding

- With enough data, fitting and generalizing **coincide at every n** (within ≈ 2).
- The shared critical width grows as $n^{1.94}$: from $0.68n$ at $n = 32$ to $1.10n$ at $n = 56$. The growth is uneven — steepest between $n = 40$ and 48 — and $n = 48$ has the largest seed spread (46.5 vs 55.8).

◆ What it means

The superlinear growth is **not** a data artefact: removing the data ceiling closed the gap but left the slope near 2. Over this range, the width a transformer needs to **learn** connectivity grows roughly quadratically in n , while the width needed to **represent** it at depth $\log N$ is sublinear — just above $\sqrt{N}$ without extra tokens (Sanford et al. 2024a). The remaining confounds are the fixed step budget (24 000 steps; more training might let narrower models get there — tested next) and the single learning rate.

The step budget: more training does not lower the critical width

Same setting at 128 000 graphs with **48 000** steps instead of 24 000, $n = 40 / 56$, on the widths around each threshold (2 seeds, 16 runs).

	critical width	24 000 steps	48 000 steps	change
$n = 40$	to fit	29.5	26.9	−2.6
	to generalize	30.4	29.8	−0.6
$n = 56$	to fit	59.4	56.3	−3.1
	to generalize	61.5	67.4 †	+5.9

Table 9: Critical width (mean of per-seed crossings at 0.95, final epoch). † seeds 62.7 / 72.0 — within the seed spread.

✓ Finding

- Twice the training lowers the width to **fit** by only ≈ 3 and leaves the width to **generalize** unchanged within the seed spread.
- At $n = 56$ the longer run starts to memorize again (48 passes over the data): $m = 64$ goes from train / test pair 0.972 / 0.960 to 0.975 / 0.948.
- The slope between $n = 40$ and 56 at 48 000 steps (≈ 2.2 fit, ≈ 2.4 generalize) is consistent with the four-point $n^{1.94}$.

◆ What it means

The width requirement is not narrower models simply learning more slowly.

The learning rate: tuning trims the slope, not the trend

Same setting at 128 000 graphs and 24 000 steps with rate 10^{-3} instead of $3 \cdot 10^{-3}$, $n = 40 / 56$, same widths (2 seeds, 16 runs). “Tuned” picks the rate per width and seed on final validation exact-match, as in Q1.

	critical width	$3 \cdot 10^{-3}$	10^{-3}	tuned
$n = 40$	to fit	29.5	28.2	28.0
	to generalize	30.4	30.1	30.2
$n = 56$	to fit	59.4	52.7	52.7
	to generalize	61.5	57.9	57.9

Table 10: Critical width (mean of per-seed crossings at 0.95, final epoch).

✓ Finding

- At $n = 40$ the rate does not matter: the curves coincide.
- At $n = 56$ the higher rate was holding the wide models back, as Q1 predicted: $m = 96$ goes from test pair 0.932 to **0.984**. Validation picks 10^{-3} for every $m \geq 48$ at $n = 56$.
- Slope between $n = 40$ and 56: $2.09 \rightarrow 1.93$. Four-point slope with the tuned values at $n = 40 / 56$ (and $3 \cdot 10^{-3}$ at 32 / 48, the only rate run there): $n^{1.85}$, seed-bootstrap 90 % range 1.68–2.03.

◆ What it means

The superlinear growth survives every confound we have tested: the learning rate (Q1 and this check), the amount of data (data check) and the training budget (step check). Over $n = 32$ –56 the width a transformer needs to **learn** connectivity grows roughly as $n^{1.9}$. $n = 32 / 48$ have not been re-run at 10^{-3} ; $n = 48$ would likely come down a little too.

8. Q6 — is the growth just the input encoding?

Each node token is its adjacency row, n numbers wide, and the model’s first layer projects it to width m . The critical width we measured is ≈ 0.7 – $1.1n$ — close to where that projection stops compressing — so the growth might come from the encoding, not the task.

Test. Feed each node $(A + I)P$ instead, with P a fixed random $n \times 96$ matrix ($\pm 1/\sqrt{96}$ entries, one “ID” vector per node, drawn per seed, never trained). The input is 96 wide for every n and still determines each neighbour set (simple decoding recovers $\approx 99\%$ of rows at $n = 32$ and 56; a 32-wide code recovers only 60–70 %, which would have confounded width with lost information). Everything else is the tuned 128 000-graph baseline: same graphs, model, 24 000 steps, rate 10^{-3} , $n = 40 / 56$, $m = 16 \dots 96$, 2 seeds (24 runs).

critical width	$n = 40$	$n = 56$	growth $40 \rightarrow 56$
baseline: adjacency rows (n wide), tuned	30.2	57.9	$\propto n^{1.93}$
Q6: projected IDs (96 wide), to generalize	37.3 (32.3 / 42.2)	73.8 (69.8 / 77.8)	$\propto n^{2.03}$
Q6: projected IDs, to fit	37.0	57.8	$\propto n^{1.33}$

Table 11: Critical width (mean of per-seed crossings at 0.95, final epoch; per-seed values in brackets).

✓ Finding

- With an input whose size does not depend on n , the width to generalize still roughly doubles from $n = 40$ to 56 ($n^{2.03}$, vs $n^{1.93}$ for the baseline).
- The projected input needs $\approx 20\text{--}25\%$ more width at both sizes: a random code of the neighbour set has to be decoded before it is usable. It shifts the curve up without changing its slope.
- Seed spread is larger than for the baseline (32 vs 42 at $n = 40$), so the two-point slope is rough; the doubling itself is clear.

◆ What it means

The growth is not an artefact of the n -wide read-in: it belongs to the task (or to the architecture beyond the first layer). Across two input encodings, learning connectivity needs width growing roughly as n^2 over this range.

9. Q7a — can depth replace width?

$n = 40$, the audited swap data (connected diameter ≈ 10), 128 000 graphs, 24 000 steps, rate 10^{-3} , depth $L \in \{2, 3, 4, 6, 8\}$, widths around each threshold, 2 seeds (60 runs).

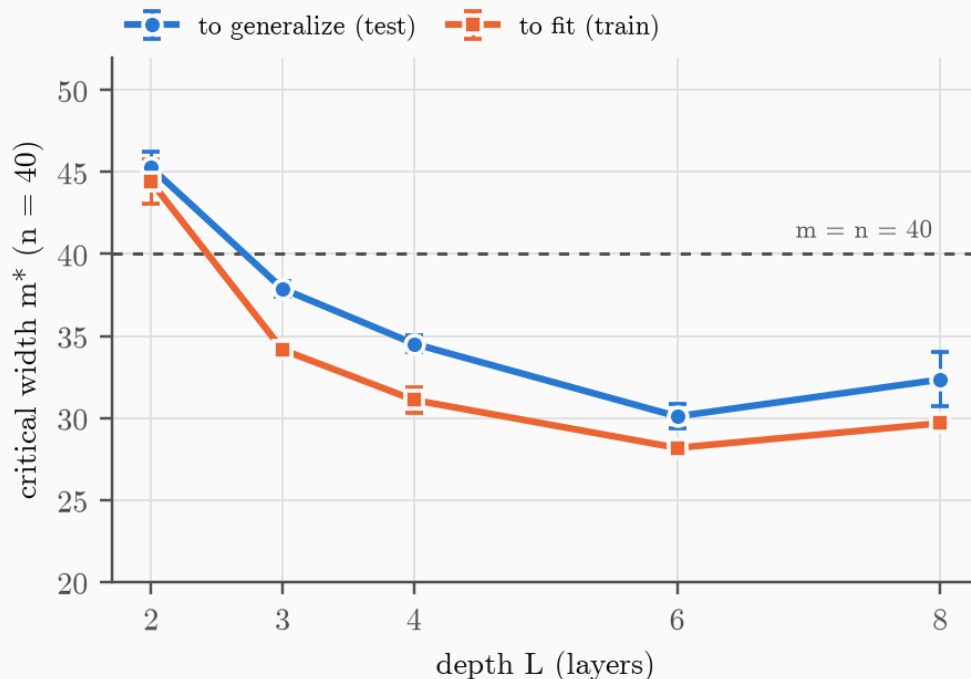


Figure 6: Critical width vs depth at $n = 40$ (error bars: range over two seeds).

depth L	2	3	4	6	8
critical width to generalize	45.3	37.9	34.5	30.1	32.3
m^*/n	1.13	0.95	0.86	0.75	0.81

Table 12: Mean of per-seed crossings at 0.95, final epoch; seeds agree within $\approx \pm 2$.**✓ Finding**

- **Two layers are enough.** At $L = 2$, $m = 48$ reaches test pair 0.968 and $m \geq 64$ reach ≈ 0.99 — on graphs of diameter ≈ 10 . The prediction “depth must cover the diameter by doubling, $2^L \geq 10$ ” was wrong. Ye et al. (2026) prove a tighter reach: an L -layer model can handle diameter up to 3^L , so two layers already cover ≈ 9 — right at our graphs’ diameter, which fits depth helping little beyond 3. (Their bound is for a restricted architecture; our standard 2-layer models still reach 0.99.)
- **At this size the trade-off is mild and saturates.** Going from 2 to 6 layers saves about a third of the width ($45 \rightarrow 30$); depth 8 is no better than 6. (Q7c: the saving grows with n .)
- **Exact reproduction.** The $L = 6$ runs repeat the Q3 learning-rate baseline at $n = 40$ and give the same critical width (30.1).

◆ What it means

With adjacency-row tokens, width $\approx n$ makes constant depth sufficient — the regime Yehudai et al. (2025) prove for their tokenization (“linear width, constant depth”); we measure the constant: $\approx 1.1n$ at depth 2. At fixed n depth matters little, yet at depth 6 the critical width grows $\approx n^2$ across sizes. Q7c asks how the depth-2 critical width grows with n .

10. Q7c — does the depth-2 width grow linearly?

Same setting as Q7a (the audited swap data, 128 000 graphs, 24 000 steps, rate 10^{-3}) at depth 2 for $n = 32 / 48 / 56$, widths 16–192 placed to cover both predictions ($m^* \propto n$ and $m^* \propto n^2$), 2 seeds (34 runs); Q7a’s depth-2 runs supply $n = 40$.

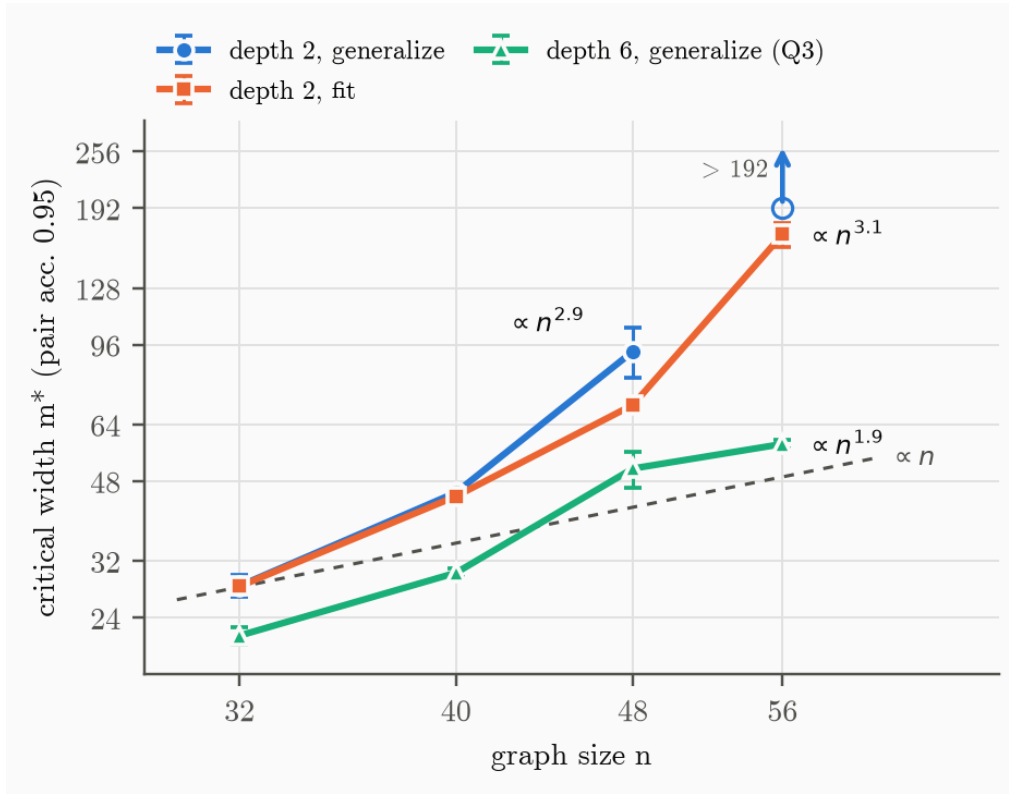


Figure 7: Critical width vs n at depth 2 and depth 6 (log-log; error bars: range over two seeds; open circle: no seed reaches 0.95 by $m = 192$). Depth 6 is Q3 at 128 000 graphs, rate 10^{-3} at $n = 40 / 56$ and $3 \cdot 10^{-3}$ at $32 / 48$.

	$n = 32$	$n = 40$	$n = 48$	$n = 56$	growth
depth 2, to fit	28.1	44.4	70.7	168.0	$\propto n^{3.07}$
depth 2, to generalize	28.2	45.3	92.7	> 192	$\propto n^{2.91}$ (to 48)
depth 6, to generalize	21.9	30.2	51.1	57.9	$\propto n^{1.85}$
depth 2 $\div$ depth 6	1.3	1.5	1.8	> 3.3	

Table 13: Critical widths (mean of per-seed crossings at 0.95, final epoch). Taking one seed per n gives fit slopes of 2.90–3.24.

✓ Finding

- **Not linear.** The depth-2 critical width grows as $\approx n^3$ — faster than depth 6’s $\approx n^{1.9}$. From $1.1n$ at $n = 40$ it reaches $3n$ at $n = 56$.
- **The robustness is in the direction, not the exponent.** With a looser threshold the fit slope is 2.4 (pair accuracy 0.90) or 2.8 (0.93): always above depth 6’s and far above linear.
- **The step to $n = 56$ is the steepest.** Local fit slopes are 2.1 ($32 \rightarrow 40$), 2.6 ($40 \rightarrow 48$) and 5.6 ($48 \rightarrow 56$). At $n = 56$ the curve is flat near the threshold — width 192 only reaches train pair 0.96 — so this point is the least certain (seeds: 158 and 178).
- **Fitting and generalizing separate again at depth 2, even with 128 000 graphs.** At $n = 56$, $m = 192$ has train pair 0.96 but test pair 0.92; at depth 6 the same data closed this gap.

◆ What it means

The constant-depth, linear-width solution exists for this tokenization (Yehudai et al. 2025), but training does not find it as n grows: two layers need ever more width than six. So Q7a’s “depth trades only mildly” holds only at $n = 40$; the width that depth saves grows with n , from $\approx 1.3\times$ at $n = 32$ to more than $3\times$ at $n = 56$. This fits the log-depth constructions (Sanford et al. 2024a): with too few layers, width has to stand in for depth at a rising cost.

11. What it all means so far

1. **Measuring width is a protocol problem first.** Learning rate and training-set size both interact with width strongly enough to fake or hide a width effect. Every later sweep tunes the rate per width and checks the data ceiling.
2. **Benchmarks decide the answer.** The same architecture needs width 8 on the leaky task and ≥ 32 –64 on the audited one at similar n . A width study is only as good as its shortcut audit — a methodological contribution in its own right.
3. **Likely new.** A novelty check (`docs/novelty-check-2026-10-08.md`) found no prior work measuring how the width needed to **learn** connectivity scales with n . The closest — Yehudai et al.’s critical width, linear in n — measures **fitting** on counting tasks at depth 1 with 5 000 graphs and a fixed learning-rate grid, exactly the confounds Q1 and Q1c expose.
4. **Representation is not learning.** Theory says width $\approx n$ at constant depth (Yehudai et al.), or width just above $\sqrt{N}$ at depth $\log N$ (Sanford et al.), is **enough to represent** connectivity. In practice the width needed to **learn** it grows roughly as $n^{1.9}$ over $n = 32$ –56 — with 128 000 graphs, with twice the training steps, with the learning rate tuned, and with a fixed-width input encoding — and the data has to grow too; at $n = 128$ nothing up to $m = 128$ learns. At depth 2, where the linear-width construction lives, it grows faster still ($\approx n^3$). This matches Saparov et al. (ICLR 2025): transformers struggle to learn search even when it is representable.
5. **A hypothesis for the n^2 .** n^2 is the number of bits in the adjacency matrix, and Yehudai et al. note that quadratic width suffices for **any** graph task by copying the whole graph into every token. SGD may be finding that brute-force solution rather than the efficient log-depth one. Q6 rules out the **input** width, not this; probing the layers for path-doubling structure (A^{2^ℓ} , as in Ye et al.) would tell.

! Caveat

- All slopes come from four graph sizes spanning less than a factor of two in n , with 2–3 seeds; local slopes between neighbouring sizes range from ≈ 1.2 to ≈ 2.9 .
- Learning-rate tuning covers $n = 40 / 56$ only; $n = 32 / 48$ use $3 \cdot 10^{-3}$.
- Exact-match gets stricter as n^2 grows; pair-accuracy thresholds are reported alongside from now on.
- The **swap** diameter grows with $\log n$, so n and path length still rise together; Q7b will separate them.
- One learning rate in the fine grid ($3 \cdot 10^{-3}$), for budget reasons; it is too high at $m = 128$, which is left out of the fit.

12. Next steps

Item	Purpose	Status
Probing	read what each layer computes: path doubling (A^{2^ℓ}) or a copy of the whole graph? — explains the n^2	next
Q7b diameter	fix n , vary the diameter: is path length or node count driving the growth?	planned
Q4 Graphormer bias	shortest-path bias gives connectivity away; needs a task it doesn’t leak	planned
Q2 task hierarchy	retrieval < connectivity < shortest path, each with its own audit	planned

Table 14: Week 6 is the first progress review (Явц 1); research freeze in week 11.

Reproduce. Every number above comes from `python width_sweep.py <sweep> --analyze` or `--curves` over `results/width/<sweep>*.jsonl` (sweeps `q1`, `q1b`, `q1c`, `q3pilot`, `q3pilot2`, `q3probe`, `q3trim`, `q3fine`, `q3data`, `q3big`, `q3steps`, `q3lr`, `q6proj`, `q7depth`, `q7c`) and `--scaling` for critical widths; the audit from `python audit_width_data.py` figures from `reports/figures/width_progress_plots.py`.